Decision driven AI development

project-kit (pkit)

Aleš Kalfas

github.com/aleskalfas/project-kit

Verze pkit 1.149 · 30. října 2026

Část 1

pkit v kostce

Proč pkit vznikl, jak ho nainstalovat a jak do sebe zapadají jeho části.

Proč pkit?

Váš AI agent pracuje rychle, ale zapomíná všechno, co jste v projektu rozhodli.

Rozhodnutí zůstávají v chatech a v hlavách lidí takže o nich agent neví a rozhoduje znovu	→ **Zapište každé rozhodnutí s důvodem** vy ho schválíte a každá session si ho přečte · Část 2
Konvence nejsou sepsané takže se agent řídí vlastními zvyky	→ **Sepište své konvence** každá session je načte a agenti se jimi řídí · Části 2, 5
Dokumentace není navázaná na kód takže po změně kódu přestane platit a nikdo si toho nevšimne	→ **Navažte dokumenty na to, na čem závisí** každá změna znovu ověří dotčené dokumenty · Část 4
Nikdo neřekne, co agent smí takže nástroj u každé akce hádá	→ **Zapište, co agent smí** rozhodnete jednou za celý projekt, v každé session stejně · Část 3

Otázka: jak ho přimět pracovat po vašem, a to bez těžkopádného frameworku?

Jak je pkit poskládaný

Skládá se z pěti vrstev, od vašich rozhodnutí až po váš kód. Každá vrstva provádí to, co určují vrstvy nad ní.



Instalace pkitu

Nástroj nainstalujete jednou, pak pkit přidáte do každého projektu jedním příkazem. Upgrade mění jen soubory pkitu.

```
# jednou, na vašem počítači
$ uv tool install \
    git+ssh://git@github.com/aleskalfas/project-kit.git@latest
$ pkit --version

# v každém projektu
$ cd your-project
$ pkit init
```



Nechcete mít pkit v repozitáři?

```
$ pkit visibility private
```

pkit zůstane jen ve vašem klonu a git ho díky souboru exclude ignoruje. Nic z něj se necommitne, a agent ho přesto používá celý.

```
$ tree -a -L 2 your-project
your-project/
├── .pkit/                          # metodika
│   ├── adapters/                   # napojení na váš AI nástroj
│   ├── agents/ core/ project/
│   ├── cli/                        # dokumentace příkazů
│   ├── decisions/ core/ project/
│   ├── lifecycle/                  # instalace, sync, upgrade
│   ├── migrations/                 # převedou projekt na nové verze
│   ├── permissions/ project/       # co agenti smějí
│   ├── process/                    # procesy se stavy, jako data
│   ├── rules/ core.md project.md   # sem píšete vy
│   ├── schemas/                    # co čtou nástroje
│   ├── scratchpad/                 # vaše nedotažené nápady
│   ├── skills/ core/ project/
│   ├── manifest.yaml               # co je nainstalováno a v jaké verzi
│   └── .gitignore                  # běhové soubory pkitu
├── .claude/                        # vaše nastavení Claude Code
│   ├── agents/  skills/            # pkitu vedle vašich
│   └── settings.json               # vaše, pkit přidá svoje
├── .agent-workspace/               # pracovní složka agentů, mimo git
├── CLAUDE.md                       # načte pravidla do každé session
└── src/                            # váš kód, pkit na něj nesahá
# fialové: pkitu, každý upgrade je nahradí · modré: vaše
```

pkit status ukáže, co pkit v projektu nastavil. Volitelné části, kterým pkit říká capabilities, přijdou později.

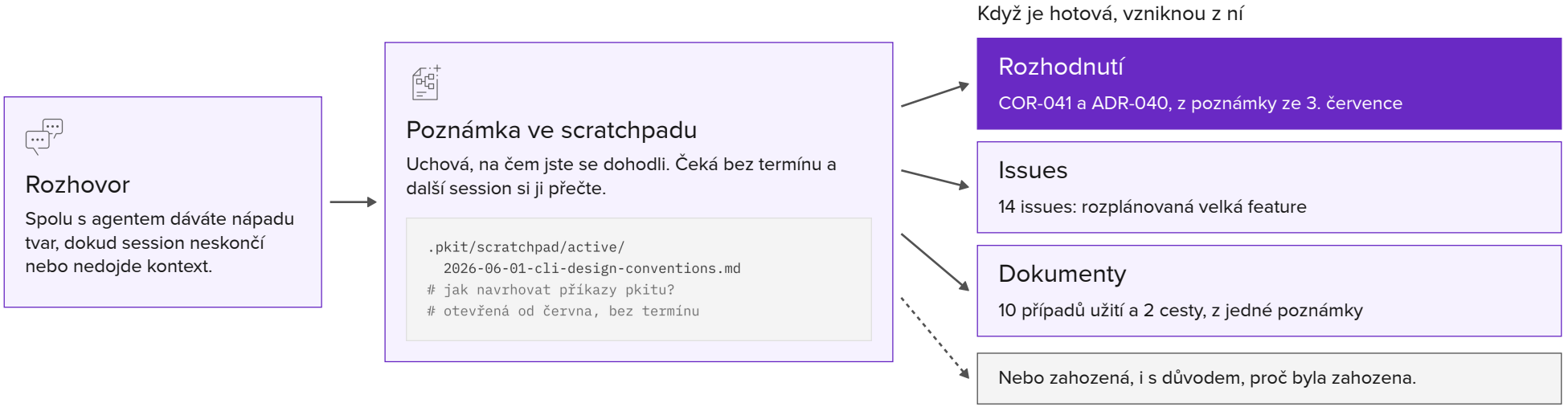
Část 2

Od nápadu k pravidlu

Kde vznikají nápady, vaše rozhodnutí, co se dozví každá session a pravidla, která hlídá program.

Kde vznikají nápady

V rozhovoru. Dohodu uchová i po konci session poznámka ve scratchpadu.
Pak z ní vzniknou záznamy, issues či dokumenty.



Omezení: Poznámka se sama nenapíše. Co zůstane jen v rozhovoru, zmizí spolu s ním.

Záznamy o rozhodnutí

Záznam o rozhodnutí uchová jednu volbu i její důvod. Podle toho, kde leží, poznáte, čeho se týká a kdo ho smí měnit.

Jak se pracuje

`.pkit/decisions/`

COR

Principy pkitu

Jak metodika funguje, stejně v každém projektu.

core/COR-065-records-and-projection.md

Leží odděleně od vašich. Každý upgrade je nahradí, proto je nikdy neupravujete.

PRJ

Váš způsob práce

Jak pracuje váš tým tam, kde pkit volbu nechává na vás.

project/PRJ-002-version-bump-policy.md

Patří jen vám. Upgrade sem nikdy nic nezapíše.

Co stavíte, ve složce, kterou si zvolíte

`docs/architecture/decisions/`

ADR

Vaše architektura

Jak je postavený váš systém: struktura, technologie a hranice.

ADR-001-project-root-resolution.md

Leží u dokumentace vašeho systému, kde se o ně stará architekt.

> Omezení: Napsat a zrevidovat záznam zabere čas. Vyplatí se jen u rozhodnutí, na kterých stojí další práce.

Rozhodnete jednou, agent to pak najde

Rozhodnutí schválíte, než na něm začne stavět práce. Později ho agent najde dřív, než změní soubory, kterých se týká.

Agent napíše návrh

Připraví záznam a naváže na něj příručku k vydávání.

```
$ cat .pkit/decisions/project/PRJ-002-….md
id: PRJ-002
title: Version-bump policy
status: proposed    # dokud ho neschválíte
## Context          # proč se to řeší
## Decision         # co se rozhodlo
## Rationale        # proč, co se zamítlo
## Implications     # co z toho plyne

# .pkit/release/README.md, jeho front matter:
  friction:
    anchors:
+     record: [PRJ-002]
```

Vy ho schválíte

Uvidíte ho ještě před mergem, dokud je změna nejlevnější.

```
# pull request, před mergem:
  .pkit/decisions/project/PRJ-002-….md
- status: proposed
+ status: accepted

  .pkit/release/README.md
    anchors:
+     record: [PRJ-002]
```

Pozdější session ho najde

Vaše důvody nezná, a tak si nejdřív vyhledá záznamy.

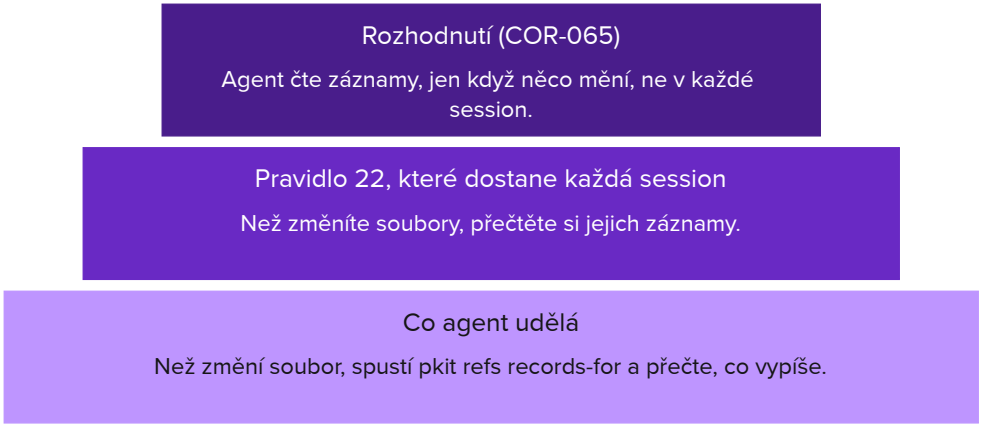
```
$ pkit refs records-for .pkit/release/README.md
6 records tied directly:
  PRJ-002  Version-bump policy …
  PRJ-004  Distribution via direct git URL …
  ADR-040  Fetch externally-sourced content …
  COR-010  Lifecycle of installed resources
  …
# agent je přečte, než soubor změní
```

> Omezení: Najde se jen záznam navázaný na soubor, nebo na dokument, který na tom souboru závisí.

Tento týden: ADR-075 určil, jak vznikají pravidla, která načítá každá session. Schválený se mergnul spolu se svým kódem.

Co se dozví každá session

Pár krátkých pravidel od pkitu i od vás. Celé záznamy by se nevešly, a tak každé pravidlo odkazuje na svůj záznam.




```
# načte se na začátku každé session
.pkit/rules/core.md      # od pkitu, v každém projektu
.pkit/rules/project.md   # váš, pkit ho nikdy nemění

# vyhledá se podle potřeby
$ pkit refs records-for <files>  # záznamy k tomu,
                                 # co se mění
$ pkit rule-sets reach <paths>   # pravidla k tomu,
                                 # co agent píše
```

> Omezení: Pravidlo je jen rada: agent ho může porušit.

Tento týden: každé pravidlo dostalo trvalé id a stav. Soubor, který načítá každá session, se z nich teď generuje.

Jak vypadají schémata a procesy

Schéma tvoří dva soubory: pravidlo jako data a tvar, který ta data musí mít.
Proces je schéma stavů a přechodů.

Schéma: dva soubory

V jádře pkitu

Pravidlo jako data

```
# .pkit/schemas/privilege-catalog.yaml
vcs-history-rewrite:
  description: Rewriting published git history (force-push).
  guardrail: true
  recognize:
    bash:
      - cmd: git
        subcommand: [push]
        flag_any: ["--force", "-f", …]
```

Tvar, který musí mít

```
# .pkit/schemas/privilege-catalog.schema.json
"entry": {
  "type": "object",
  "additionalProperties": false,
  "required": ["description", "recognize"],
  "properties": {
    "description": {
      "type": "string",
      "minLength": 1,
```

Proces: zvláštní druh schématu

Příklad z capability: není v jádře pkitu

Workflow pro issues

```
# .pkit/capabilities/project-management/
#   schemas/workflow.yaml
process:
  id: issue-lifecycle
  states:  # todo … done, celkem pět
  transitions:
    - from: review
      to: done
      authorisation: user
```

Co dodá jádro pkitu, takže to žádná capability ani projekt nestaví znovu

```
# pro schémata
$ pkit new schema <capability> <name>   # pravidlo a jeho tvar, v páru
$ pkit validate                         # ověří tvar každého souboru
$ pkit data validate <file>             # i vaše vlastní datové soubory
```

```
# pro procesy
$ pkit process new <capability>:<id>    # proces, gate zatím jen jako stub
$ pkit process status                   # v jakém je stavu a proč
$ pkit process move                     # přechod, jen když je povolený
```

> Omezení: pkit validate ověří jen tvar souboru. Jestli je pravidlo v něm správné, neověří.

V čisté instalaci: 8 schémat pkitu a tvar, který musí mít každý proces. Všechno to zkontroluje pkit validate.

Jak capability používá proces

project-management přináší workflow pro issues. Přechody hlídá engine pkitu, takže capability vlastní engine nemá.



Přechod ve workflow.yaml z capability

```
# workflow.yaml z project-management
- from: in-progress
  to: done
  authorisation: user
  gate:
    kind: deterministic
    predicate:
      run: gate-checkboxes-ticked
```

Tvar souboru určuje jádro pkitu, pro každý proces stejně.

Na co se agent ptá pkitu

```
$ pkit process status \
    project-management:issue-lifecycle
    --subject 2270
Where: in-progress — Branch open, work in flight.
Legal moves (live precheck):
  ✗ done  [close-issue]
      7 unticked checkbox(es) remain
```

Proč to pomáhá

Agent se zeptá, jak je práce daleko, místo aby hádal.

Přechod projde jen po kontrole nebo s vaším ano. Slovo agenta nestačí.

Capability si vlastní engine nepsala. Každý přechod za ni provede engine pkitu.

Omezení: pkit dodává engine, ale žádný proces. Ten jediný, který se dnes používá, přináší capability project-management.

V project-kitu dnes: Feature #2270 nejde uzavřít, dokud v ní zbývá 7 nezaškrtnutých políček.

Část 3

Jak agent pracuje

Postupy, role a co smí každý agent.

Odkud se berou skilly

Z rozhodnutí pkitu, která zavádějí druhy souborů. Co vyžaduje úsudek, řeší skill. Co musí být přesné, udělá příkaz.

Rozhodnutí zavede druh souboru	Skill píše podle rozhodnutí	Příkaz vytvoří soubor	Co dostanete nový soubor k vyplnění
COR-012 Poznámky	scratchpad-author	`pkit new scratchpad`	Poznámka s datem v active/
COR-006 Záznamy o rozhodnutí	decision-author	`pkit new decision`	Záznam s dalším číslem čeká, až ho schválíte
COR-013 Agenti	agent-author	`pkit new agent`	Soubor agenta podle vzoru pkitu

Proč pkit práci takhle dělí

Důvod je zapsaný na jednom místě. Skill na něj odkazuje a nikdy ho nekopíruje.	Příkaz nevybere název ani nenapíše obsah. Bez skillu agenti pravidla psaní vynechávali.	Agent se může přepočítat. Záznam psaný ručně jednou dostal špatné číslo.	Každý soubor daného druhu má stejný tvar, ať ho píše kdokoli.

Omezení: Skilly pkitu pokrývají jen jeho vlastní soubory. Na práci, která se ve vašem projektu opakuje, si napíšete vlastní.

V project-kitu: pkit dodává 11 skillů a každý má svůj příkaz.

Skill krok za krokem

Agent prochází kroky popořadě. V kroku 6 spustí příkaz, takže číslo i struktura souboru vždycky sedí.

Skill decision-author: nový záznam o rozhodnutí

1	Ověřit, že jeho rozhodnutí jsou schválená	
2	Vybrat druh záznamu	
3	Přečíst pravidla psaní	
4	Přečíst strukturu záznamu	
5	Vybrat krátký název	
6	Vytvořit soubor	jeho příkaz
7	Napsat obsah	
8	Zkontrolovat podle pravidel	
9	Ukázat vám návrh	
10	Commitnout, až ho schválíte	

Jeho příkaz: pkit new decision

```
$ pkit new decision project test-coverage-floor
Stamped: .pkit/decisions/project/PRJ-012-test-coverage-floor.md
# v souboru:
id: PRJ-012          # další číslo, nikdy odhadem
status: proposed     # dokud ho neschválíte
## Context
## Decision
## Rationale
## Implications
```

> Omezení: Nic nekontroluje, jestli agent skill použil. Příkaz vytvoří soubor i bez skillu.

Session zná ze skillů jen jejich krátké popisy. Kroky si načte, až na úkol dojde.

Odkud se berou agenti

Když se stejná kontrola pořád vrací, rozhodnutí z ní udělá roli. Ta má jednu práci a jen nástroje, které potřebuje.

Rozhodnutí zavede roli	Agent má jeden úkol	Jeho meze jen potřebné nástroje	Co zachytí abyste nemuseli vy
COR-024 Druhý názor	critic	Jen čte	Díry v návrhu, než ho přečtete
COR-024 Strážce celého systému	architect	Mění jen dokumenty architektury	Změnu, která rozbije celek
COR-057 Jeden reviewer na pravidla	rule-set-reviewer	Nikdy nemění, co kontroluje	Porušené pravidlo i s řádkem

Proč má pkit agenty			
Rozhodnutí z každé kontroly udělá práci jedné role. Jinak celý systém nikdo nehlídá.	Agent, který píše, souhlasí sám se sebou. Druhý agent ne.	Reviewer, který nemůže upravovat, zůstává nezávislý.	Chyba zachycená tady stojí čas agenta, ne váš.

Omezení: Které nástroje agent smí, hlídá AI nástroj. Které soubory smí měnit, je jen pravidlo, které dodržuje. Zámek to není.

V project-kitu: pkit dodává 6 agentů. Každý je soubor se svou prací, nástroji a rozhodnutími, kterými se řídí.

Kdo práci uvidí a kdy

Každá role ji vidí ve svou chvíli. Critic vidí návrh dřív než vy. Revieweři kontrolují až hotovou změnu.



Omezení: Každé kolo review stojí několik běhů agentů.

S čím přichází každý agent

S vazbami na svá rozhodnutí a s cestami do vašich složek. Cesty doplní pkit sám a každou vazbu zkontroluje.

Agent pkitu

```
# .pkit/agents/core/architect.md
gates: [COR-024, COR-025, COR-014]
owns:
  - <architecture-docs>
  - <adr-records>
```

+

Vaše cesty

```
# .pkit/agents/project/overlay.yaml
architecture-docs:
  - README.md
adr-records:
  - docs/architecture/decisions/
```

→

Co dostane váš AI nástroj

```
# .claude/agents/architect.md
gates: [COR-024, COR-025, COR-014]
owns:
  - README.md
  - docs/architecture/decisions/
```

Jeho rozhodnutí

V prvních řádcích jmenuje rozhodnutí, kterými se řídí, a dokumenty, které čte.

Ověřeno

pkit validate ověří každé jméno i to, že dva agenti nevlastní stejný soubor.

Vždy aktuální

Každý upgrade agenta nasadí znovu, s vašimi cestami.

Musíte agenty zapojit?

```
$ pkit sync   # po změně overlay.yaml
```

Ne. Čistá instalace doplní výchozí cesty a všech šest agentů hned funguje.

Omezení: Agenty pkitu nikdy neupravujete. Na jinou roli si napíšete vlastního agenta do .pkit/agents/project/.

V čisté instalaci pkit nasadil všech 6 agentů najednou, s výchozími cestami. pkit validate zkontroloval 17 skillů a agentů.

Odkud se berou oprávnění

Z pravidel, která projekt zapíše jako data. AI nástroj je uplatní při každém volání, i když na ně agent zapomene.

Pravidlo jak rozhodlo COR-028	→ Zapsáno jako data v čitelných souborech	→ Dostane AI nástroj a sám uplatní	→ Na co volání narazí pokaždé
Pojistky pro vás i každého agenta	rm -r, sudo a force-push, zakázané všem	Zákaz v jeho nastavení	Odmítnuto v každém režimu
Vlastní nástroje agenta v souboru agenta	critic: Read, Glob, Grep, WebFetch	Seznam nástrojů toho agenta	Nespustí příkaz, nezapíše soubor
Pravidlo agenta z project-management	project-manager: žádné změny přímým gh	Hook, před každým voláním	Odmítnuto, a tak použije příkazy pkitu

Proč pkit oprávnění zapisuje

Pravidlo, které agent čte, je rada. Pravidlo, které uplatní AI nástroj, už ne.	Zapsané pravidlo se dá přečíst a projít v review jako každá jiná změna.	Nastavení platí pro všechny agenty stejně. Hook rozhoduje u každého zvlášť.	Příkazy pkitu mají kontroly, které přímé příkazy přeskočí. Agent je neobejde.

> Omezení: Zákaz zastaví jen ty podoby příkazu, které pkit pozná. Příkaz schovaný za jiným může proklouznout.

V project-kitu: agent project-manager nesmí nic měnit přímým příkazem gh. Používá příkazy pkitu, které issue nejdřív ověří.

Oprávnění od prvního dne

Čistá instalace odmítá destruktivní příkazy a povoluje běžné. Zbylé dotazy na povolení proto stojí za přečtení.

Nastavení vašeho AI nástroje po instalaci

```
# .claude/settings.json
"deny": [
  "Bash(rm -rf:*)", "Bash(sudo:*)",
  "Bash(git push --force:*)",
  "Bash(git reset --hard:*)", …
"allow": [
  "Bash(git:*)", "Bash(gh:*)", "Bash(pkit:*)",
  "Read", "Edit", "Skill(decision-author)", …
```

Co k tomu říká pkit

```
$ pkit permissions explain
all  (every agent and the operator)
  deny  destructive-fs        guardrail
  deny  privilege-escalation  guardrail
  deny  vcs-history-rewrite   guardrail
```

Zakázáno všem

Rekurzivní mazání, sudo a force-push nespustí nikdo, ani vy, v žádném režimu.

Běžné bez ptaní

Čtení, git, gh, pkit a skilly pkitu běží bez dotazu.

Vždy aktuální

Každý upgrade znovu doplní pravidla pkitu a vaše nechá, jak jsou.

Agenti bez dozoru?

```
$ pkit permissions setup autonomy
```

Přidá profil, hook a sandbox OS (kromě macOS) a výsledek ověří.

Omezení: Bez hooku nejde pravidlo uplatnit jen na jednoho agenta. Nastavení platí pro všechny agenty stejně.

Čistá instalace, ještě bez vašich pravidel, odmítne 14 podob 4 nebezpečných příkazů a povolí 37 běžných.

Část 4

Udržet vše v souladu

Co pkit kontroluje a dokumenty, které nezastarají.

Co pkit kontroluje a proč

Celý projekt a každou změnu před mergem. Agent totiž slepě věří všemu, co najde v souborech.

Celý projekt

```
$ pkit validate
  manifests · schemas · configuration
  packages · connections · versions
  decisions · friction · rule-sets
  refs · process · data
  12 validator(s) ran; 0 error(s)
  All checks passed.
```

Každá změna před mergem

```
$ pkit friction check
  # dokument navázaný na změněný soubor,
  # bez odpovědi
$ pkit decisions check-diff
  # číslo záznamu nebo bodu
  # použité dvakrát nebo změněné
```

Lidé zapomínají

Změna upraví jeden soubor. To, co na něm závisí, leží jinde.

Review to nevidí

Reviewer čte jen samotnou změnu. Soubor, který kvůli ní zastaral, nevidí.

Agenti věří souborům

Agent jedná podle souboru. Kontrola ověří i to, o čem agent nepochybuje.

Při každém pushi?

```
$ pkit validate && pkit friction check
```

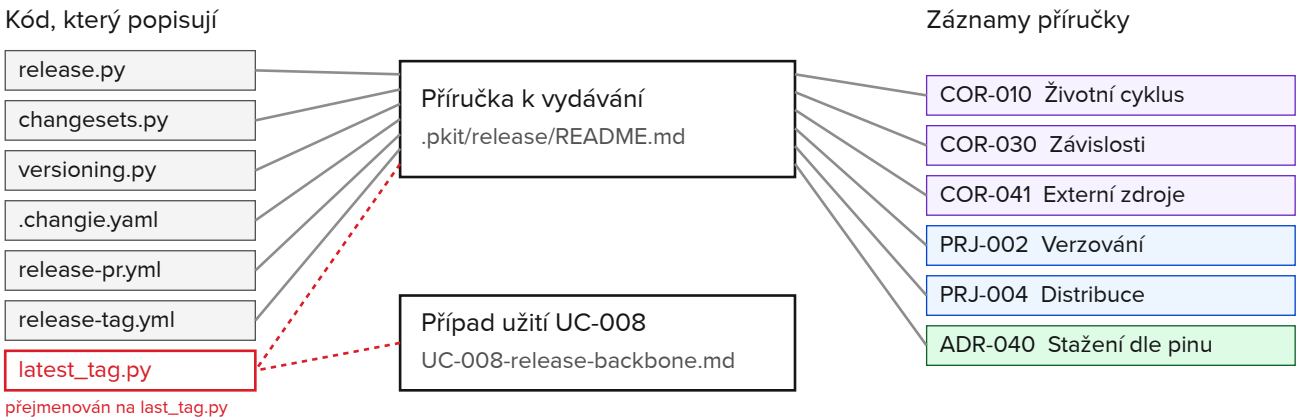
pkit nepřidá git hook ani CI. Zapojte si oba příkazy sami, jako project-kit.

Omezení: Kontrola najde, co je rozbité nebo bez odpovědi. Jestli dokument stále platí, posoudí reviewer.

Hned po instalaci spustí pkit validate 12 kontrol a nemusíte nic nastavovat. Každá capability přidá další, v project-kitu je jich 16.

Vazba pod kontrolou

Přejmenujte soubor a kontrola vám ještě před mergem řekne, které dokumenty jsou na něj navázané.




```
$ git mv scripts/latest_tag.py scripts/last_tag.py
$ pkit friction check
FINDINGS
  .pkit/release/README.md
    dead-anchor  path scripts/latest_tag.py
      matches no file; the diff removed or
      moved its target
  tech-docs/analysis/…/UC-008-release-backbone.md
    dead-anchor  path scripts/latest_tag.py
Result: failed   (2 dead anchors)
# změna neprojde, dokud vazby nevedou na last_tag.py
```

> Omezení: Kontrola ověří, že každá vazba někam vede. Jestli dokument stále platí, nepozná.

V project-kitu běží tahle kontrola v pre-push hooku a na každém pull requestu. Zapojil ji tam sám projekt.

Dokumentace, která pozná, že už neplatí

Když se změní to, na čem dokument závisí, musí se k tomu změna vyjádřit: upraveno, stále platí, nebo odloženo.

Kontrola na dokument upozorní

```
# změna přesune changesety do .changesets/
$ pkit friction check
  .pkit/release/README.md
    friction  path .changie.yaml  changed in this diff
              and carries no answer
Result: failed

# po odpovědi
$ pkit friction check
    answered  path .changie.yaml  changed in this diff;
              answered: updated
Result: passed
```

Změna odpoví jedním ze tří způsobů

Upraveno	Změna opravila i dokument.
Stále platí	V dokumentu nic nepřestalo platit. Odpověď řekne proč.
Odloženo	Odpověď uvede issue, které dokument opraví.

```
# .pkit/project/friction/answers/20261030T024530Z-ba8d160a.yaml
answers:
  - artefact: .pkit/release/README.md
    revalidated:
      outcome: updated
```

Proč pkit chce odpověď

Ne každá změna dokument zneplatní. Odpověď to rozliší.

Každou odpověď ověří reviewer. „Stále platí“ tak musí mít pádný důvod.

Odložená oprava zůstane v evidenci i s issue, které ji vyřeší.

> Omezení: Kontrola hlídá jen dokumenty, které vyjmenujete i s tím, na čem závisí. Ostatní zastarávají jako dřív.

V project-kitu dnes čeká 21 odpovědí „odloženo“. pkit friction debt vypíše každou i s důvodem.

Část 5

Co si doinstalujete

Volitelné části, kterým pkit říká capabilities: co se hodí k vaší práci a jak se propojují.

Proč má pkit capabilities

V jádru pkitu je jen to, co potřebuje každý projekt. Zbytek tvoří volitelné části, kterým pkit říká capabilities.

Jádro pkitu: v každém projektu	Capability: jedna složka k instalaci	
Rozhodnutí	Záznamy o rozhodnutí	`decisions/`
Pravidla · schémata · procesy	Schémata a procesy	`schemas/`
Skilly · agenti · oprávnění	Skilly, agenti, práva	`skills/ agents/ permissions/`
Příkazy a kontroly	Příkazy a kontroly	`scripts/ package.yaml`

Jediný příkaz ho zapojí (výpis z čisté instalace)

```
$ pkit capabilities install project-management
  Installed capability 'project-management' v0.54.0
  merged   .claude/settings.json      # povolí jeho skill
  created  .claude/skills/pm/SKILL.md     # a 6 postupů
  created  .claude/agents/pm-reviewer.md
  created  .claude/agents/project-manager.md
$ pkit permissions explain
agent:project-manager
  deny  issue-tracker-write
  (contributed by capability: project-management)
$ pkit validate
  13 validator(s) ran; 0 error(s)     # předtím 12
  All checks passed.
```

Proč má pkit capabilities

Jádro má jen to, co potřebuje každý projekt. Zbytek by jen zatěžoval.

Capability dostane každý upgrade jako jádro. Kopie souborů ne.

Jedna složka: z cesty poznáte, čí soubor je.

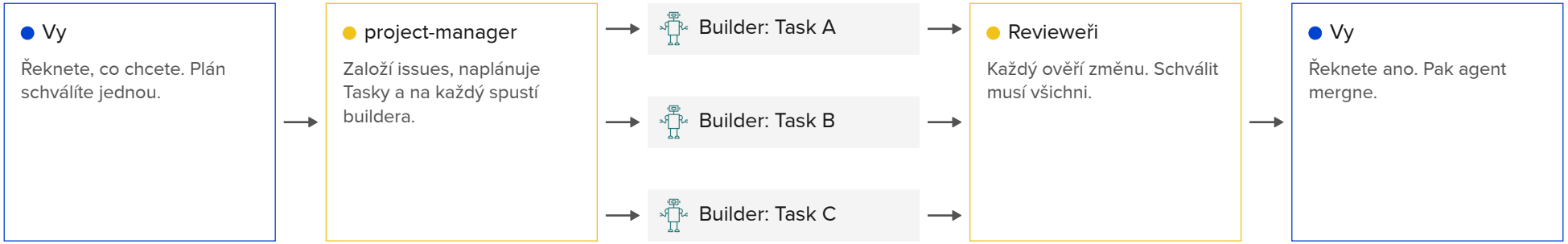
Capability vzniká jen z potřeby, která se v praxi vracela.

Omezení: Některé capabilities závisí na jiných: software-engineering se bez project-management nenainstaluje.

pkit 1.149 dodává 4 capabilities, další 3 přijdou s verzí 2.0. Jedna se do čistého projektu nainstalovala jedním příkazem za 8 sekund.

Co přináší project-management

Jednoho agenta, který založí issues, naplánuje práci a souběžně spustí buildery. Vy schválíte plán a každý merge.



Proč rozhodujete na začátku i na konci: plán a merge za tým schvalujete jen vy. Práci mezi tím udělá agent.

Jeho skill a agenti

Skill pm, agent project-manager a pm-reviewer, který projde každý pull request.

88 příkazů

pkit pm … zakládá, posouvá a merguje. Každý krok nejdřív ověří.

Workflow pro issues

Každý přechod ověří engine pkitu.

Co uděláte

Dejte mu GitHub repozitář a jednou spusťte pkit pm bootstrap.

Omezení: Umí jen GitHub a je velký: 63 záznamů o rozhodnutí a 88 příkazů. Stačí ale mluvit s agentem, manuál číst nemusíte.

V project-kitu 29. října: 22 mergnutých pull requestů, až 7 otevřených najednou, každý s verdikty svých reviewerů.

Co přináší software-engineering

Buildera, který dodržuje vaše konvence, a tři reviewery, kteří zkontrolují každou změnu kódu před mergem.

Jeho agenti po instalaci

```
$ pkit capabilities install software-engineering
  created  .claude/agents/software-engineer.md
  created  .claude/agents/code-reviewer.md
  created  .claude/agents/security-reviewer.md
  created  .claude/agents/docs-reviewer.md
# software-engineer: vyvíjí podle vašich konvencí
# code-reviewer: je kód správně?
# security-reviewer: tajné údaje, injekce
# docs-reviewer: platí ještě dokumentace?
```

+

Kdo musí schválit merge

```
# review-contributions.yaml
- floor: touches-code
  reviewer: code-reviewer
- floor: touches-code
  reviewer: security-reviewer
- floor: touches-code
  reviewer: docs-reviewer
- match: {type: "*"}
  reviewer: docs-reviewer
```

→

Merge pull requestu #2279

```
# první kolo
pm-reviewer        APPROVED
docs-reviewer      APPROVED
code-reviewer      CHANGES_REQUESTED
  [block] The new Status writer
  can't find an org-owned board …
security-reviewer  APPROVED
rule-set-reviewer  APPROVED
# opraveno, schválilo všech pět, mergnuto
```

Vaše konvence

Napíšete je jednou a builder si je přečte nejdřív. Zůstanou vaše i bez capability.

Kód projde review

Dřív pkit kód nekontroloval. Mergla se tak i změna se dvěma bezpečnostními dírami.

Blokace má váhu

Reviewer blokuje, jen když umí chybu ukázat. Všechno ostatní je rada.

Co uděláte

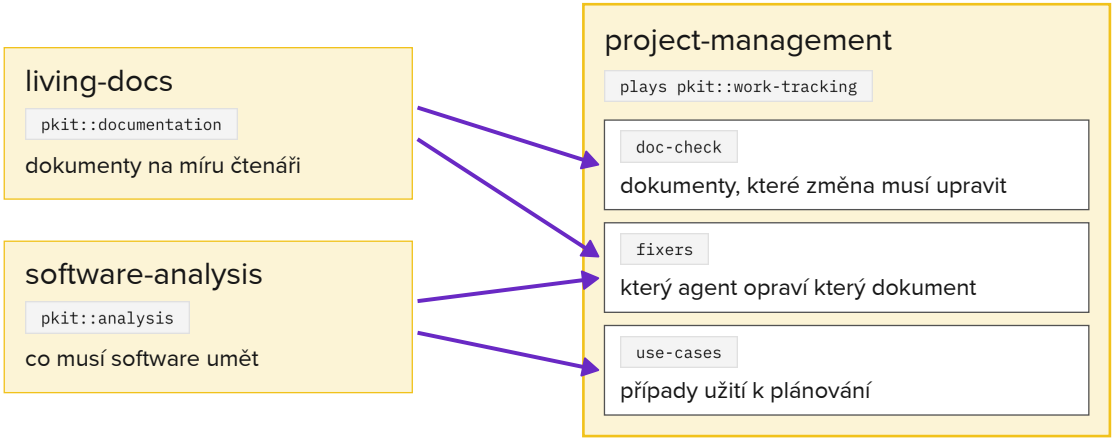
Sepište své konvence a jejich soubor uveďte v overlay.yaml.

> Omezení: Když konvence sepsané nemáte, builder i revieweři se drží obecné dobré praxe.

V project-kitu 29. října: 21 z 22 mergů měnilo kód a každý z nich prošel reviewery kódu, bezpečnosti a dokumentace.

Jak capabilities spolupracují

Capability nikdy nejmenuje jinou capability, jen roli. Když něco potřebuje, otevře slot. Vyplní ho kterákoli jiná.



Co vypíše pkit status v project-kitu

```
$ pkit status
  Connections
    roles   3 named: 3 active, 0 in conflict
      pkit::analysis      → software-analysis
      pkit::documentation → living-docs
      pkit::work-tracking → project-management
    points  6 defined: 2 unfilled
    pkit::work-tracking:fixers
      data v1 · project-management — filled
      bound  software-analysis (contributes)
      bound  living-docs (contributes)
```

Proč pkit propojuje capabilities takhle

Capability vyměníte za jinou ve stejné roli. Zbytek zůstane, jak je.

Instalujete jen to, co potřebujete. Prázdný slot ničemu nevadí.

pkit status zapojení ukáže a pkit validate ho zkontroluje.

> Omezení: Každou roli zatím hraje jen jedna capability. Výměna je tedy navržená, ale ještě nevyzkoušená.

V project-kitu: 3 role, každou hraje jedna capability. Ze 6 slotů jsou 4 vyplněné.

Otázky?

Všechno, co jste viděli, je v repozitáři project-kitu. Ten se vyvíjí právě pomocí pkitu.